ĐẠI HỌC QUỐC GIA HÀ NỘI — TRƯỜNG ĐẠI HỌC CÔNG NGHỆ | KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP CÁ NHÂN 3 (BT3) — BIẾN THỂ VT-099

Đề tài: AI-07 — Explainable Course Recommender & RAG Tra cứu Đào tạo HUS

Sinh viên: **Thái Văn Thắng** — MSSV: **23001939**

Nhóm bài tập lớn: SE2026-T25

Baseline Commit: `1205fa94eadfaa6a52721e85cdab8a7dea9cfa11`

Nhánh Git: `bt3/23001939-vt099` (GitHub Remote)

> ***Cam kết danh dự:*** *"Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI."*

> **Lưu ý tích hợp:** Không merge vào main khi nhóm chưa review.

1. Specification, Constraints & Invariants

Lecture 06 & 07

1.1. Problem Statement & Business Context

Trong hệ thống RAG tra cứu đào tạo HUS, khối lượng dữ liệu crawl từ các website thành viên ngày càng mở rộng. Ban quản trị cần phân quyền cho một vai trò chuyên biệt là `reviewer` **(người kiểm duyệt)** nhằm san sẻ việc thẩm định tài liệu mà không cấp quyền quản trị tối cao. Vai trò reviewer có quyền đọc danh sách, xem chi tiết và đưa ra quyết định phê duyệt (`approved`) hoặc từ chối (`rejected`) tài liệu đang ở trạng thái `pending`. Tuy nhiên, theo nguyên tắc đặc quyền tối thiểu (Least Privilege), **reviewer bị nghiêm cấm tuyệt đối việc sửa đổi siêu dữ liệu gốc** (tiêu đề, ngành, loại tài liệu qua endpoint `PUT /admin/crawler/documents/{id}/metadata` $\to$ HTTP 403 Forbidden).

Đồng thời, mã nguồn hiện tại của dịch vụ kiểm duyệt (`app/crawler/document_review_service.py`) dù đã có khóa dòng `SELECT ... FOR UPDATE` và `pg_advisory_xact_lock` nhưng **chưa có cơ chế so khớp trạng thái kỳ vọng (Compare-and-Set - CAS)**. Khi hai reviewer cùng mở một tài liệu `pending` và gửi đồng thời hai quyết định trái ngược nhau (A duyệt `approve`, B duyệt `reject`), hệ thống sẽ âm thầm chấp nhận cả hai và ghi đè quyết định của người đến sau (lỗi **Last-Writer-Wins / Conflicting Concurrent Modification**), làm mất tính toàn vẹn nghiệp vụ và sai lệch audit trail.

1.2. Acceptance Criteria (Kiểm thử được & Strong Oracles)

- **AC1 (Quyền đọc & duyệt của Reviewer):** Tài khoản `role='reviewer'` có session hợp lệ được phép truy cập `GET /admin/crawler/documents`, `GET /admin/crawler/documents/{id}`, `POST ... /approve`, `POST ... /reject` và nhận phản hồi **HTTP 200 OK**.
- **AC2 (Bảo vệ dữ liệu gốc - Cấm sửa Metadata):** Reviewer bị cấm tuyệt đối (**HTTP 403 Forbidden**) khi gửi yêu cầu `PUT /admin/crawler/documents/{id}/metadata`. Chỉ `admin` mới có quyền này. *(Được kiểm chứng qua test chống pass rỗng: xác nhận reviewer gọi GET 200 thành công trước khi gọi PUT bị 403).*
- **AC3 (Xử lý xung đột đồng thời qua Compare-and-Set):** Endpoint duyệt hỗ trợ trường tùy chọn `expected_status`. Khi 2 yêu cầu đồng thời cùng gửi `expected_status="pending"` vào tài liệu `pending`: đúng 1 yêu cầu thành công (HTTP 200), yêu cầu đến sau phát hiện trạng thái đã đổi và bị từ chối với **HTTP 409 Conflict**. Nếu không gửi `expected_status`, hệ thống giữ nguyên hành vi duyệt lại vô điều kiện.
- **AC4 (Tính giải trình và toàn vẹn Audit Log):** Quyết định của reviewer phải lưu đúng `reviewer_id` vào bảng `crawled_documents` và phát sinh một bản ghi kiểm toán trong bảng `admin_audit_logs`.

1.3. Non-functional Constraints & Invariant Cốt lõi

- **NFC1 (Hiệu năng tải đồng thời):** Hệ thống phải xử lý ổn định **80 tác vụ đồng thời** với độ trễ **p95 không vượt quá 2.50 giây**, giải thích rõ cơ chế điều phối của SQLAlchemy `QueuePool` và `pg_advisory_xact_lock`.
- **NFC2 (Bảo toàn 100% tương thích ngược):** Giữ nguyên schema bảng `crawled_documents`; bảo toàn 100% kết quả test cũ (đặc biệt tính năng duyệt lại tài liệu trong `test_admin_document_review.py`).
- **Core Invariant:** *"Khi client chỉ định trạng thái kỳ vọng `expected_status`, tài liệu chỉ được chuyển trạng thái kiểm duyệt nếu trạng thái hiện tại khớp chính xác với kỳ vọng. Mọi yêu cầu đồng thời đến sau có cùng kỳ vọng phải bị từ chối với HTTP 409 Conflict (loại trừ Last-Writer-Wins khi client chỉ định kỳ vọng). Vai trò reviewer chỉ có quyền đọc và duyệt, tuyệt đối không được phép chỉnh sửa dữ liệu gốc metadata."*
- **Non-goals, Residual Risks & Schema Governance:**
  - *Phạm vi CAS:* CAS là cơ chế tùy chọn khi client truyền `expected_status`. Client cũ không truyền vẫn được cập nhật vô điều kiện để bảo toàn 100% tính tương thích ngược duyệt lại (NFC2).
  - *Giới hạn Tạo Reviewer:* `app/api/auth.py:179` hiện chỉ cho phép cập nhật vai trò giữa `student` và `admin`; việc tạo tài khoản `reviewer` trong hệ thống hiện thực hiện qua seed database hoặc script quản trị, chưa hỗ trợ trực tiếp từ UI quản lý người dùng.
  - *Quy chuẩn Schema DB & Giới hạn Downgrade:* Đã bổ sung migration Alembic `database/hus_alembic/versions/20261111_14_reviewer_role.py` (`hus_20261111_14`) mở rộng ràng buộc `accounts_role_check` cho `reviewer`, loại bỏ hoàn toàn rủi ro *false fixtures* do chạy DDL trong test. Lưu ý khi `downgrade()`, cần dọn dẹp hoặc chuyển role các tài khoản reviewer trước để tránh lỗi vi phạm ràng buộc check constraint.
  - *Yêu cầu Môi trường:* Database kiểm thử (`HUS_TEST_DATABASE_URL`) phải được migrate lên head `hus_20261111_14` trước khi chạy test suite.

2. AI-Assisted Implementation Plan & Architectural Trade-offs

Lecture 06

2.1. Đánh giá 4 Phương án Kiến trúc (Options Comparison)

Phương án A: CAS Tùy chọn qua Payload + FOR UPDATE — ĐƯỢC CHỌN

Cơ chế: Thêm `expected_status: str | None = None` vào schema request; trong giao dịch, sau khi lấy khóa hàng `FOR UPDATE`, kiểm tra `if expected_status and doc['review_status'] ≠ expected_status: raise DocumentConflictError` $\to$ HTTP 409 Conflict.

Ưu điểm & Evidence: Không đổi schema DB; diff nhỏ gọn (Lecture 06 tr.13); bảo toàn 100% test cũ (`test_admin_document_review.py` dòng 98-116 duyệt lại tài liệu sau khi reject); loại trừ triệt để Last-Writer-Wins khi client truyền kỳ vọng.

Phương án B: Khóa Cứng Unconditional Pending-Only — BỊ LOẠI

Cơ chế: Sửa `review_document()` bắt buộc chỉ cho phép duyệt tài liệu đang có trạng thái `pending`. Nếu đã khác `pending` thì ném lỗi.

Lý do bác bỏ: Phá vỡ trực tiếp tính năng nghiệp vụ duyệt lại tài liệu và làm FAIL test regression hiện có trong repo (`tests/api/test_admin_document_review.py` dòng 107-116), vi phạm nghiêm trọng ràng buộc NFC2.

Phương án C: Optimistic Locking với Cột `version` — BỊ LOẠI

Cơ chế: Thêm cột `version INT DEFAULT 1` vào bảng `crawled_documents`, kiểm tra `UPDATE ... WHERE id=:id AND version=:v`.

Lý do bác bỏ: Đòi hỏi migration schema DB, sửa toàn bộ repository, model và API contract. Vi phạm nguyên tắc vertical slice nhỏ nhất (Lecture 06 tr.13) và rủi ro hồi quy cao.

Phương án D: In-Memory Mutex Lock (`threading.Lock`) — BỊ LOẠI

Cơ chế: Dùng khóa bộ nhớ `threading.Lock()` trong tiến trình Python để tuần tự hóa các yêu cầu duyệt.

Lý do bác bỏ: Theo Lecture 07 tr.10, integration test phải kiểm chứng ranh giới App $\leftrightarrow$ DB. Khóa bộ nhớ hoàn toàn vô hiệu khi ứng dụng chạy nhiều Uvicorn worker hoặc nhiều container phân tán.

2.2. Phản biện và Sửa đổi Đề xuất AI dựa trên Bằng chứng Codebase Thực tế

Hạng mục AI Đề xuất	Phát hiện Lệch lạc của Kỹ sư	Hiệu chỉnh & Bằng chứng Repository
1. URL Endpoint `POST /admin/documents/{id}/review`	AI phỏng đoán endpoint generic không có trong codebase.	Đọc `app/api/admin.py` dòng 847, 889: Endpoint thực là `.../approve` và `.../reject`; sửa metadata là `PUT .../metadata` (dòng 807).
2. Khóa Giao dịch AI nhận định "chưa có khóa DB"	AI ảo giác, đề xuất thêm `FOR UPDATE` và khóa mutex ngoài.	Đọc `document_review_service.py` dòng 479-511: Code đã có `publication_lock` và `FOR UPDATE`. Nguyên nhân gốc là thiếu CAS `expected_status`.
3. Phạm vi Auth Chỉ sửa câu SQL `PRINCIPAL`	Reviewer không đăng nhập được vì `login()` vẫn chặn.	Đọc `app/auth/repository.py` dòng 91 & 102: Mở rộng cả câu query đăng nhập và điều kiện kiểm tra role trong `login()` cho `reviewer`.
4. Xử lý Mã lỗi HTTP Bắt `ValueError` trả về 400	Lỗi xung đột bị nuốt thành 400 Bad Request thay vì 409 Conflict.	Trong `admin.py`, đặt `except DocumentConflictError` trước `except ValueError`, ánh xạ chuẩn xác thành HTTP 409 (Strong Oracle).
5. Tương thích Nghiệp vụ Đề xuất cấm duyệt lại (Option B)	Làm gãy test regression duyệt lại tài liệu đã reject.	Chuyển sang Option A: CAS chỉ kích hoạt khi client truyền `expected_status`. Client cũ không truyền vẫn duyệt lại bình thường.
6. Bảo vệ Git Tree Tác dụng phụ ghi file khi test	Hàm `export_git_reviews_from_db` ghi đè file `document_reviews.json` làm bẩn working tree.	Monkeypatch hàm export trong fixture test API để cô lập tuyệt đối, giữ sạch working tree cho Git PR.

3. Risk-Based Test Strategy & Test Case Matrix

Lecture 07

3.1. Phân bổ Cấp độ Kiểm thử (Test Portfolio) & Lý do Loại trừ E2E

Theo Lecture 07 (tr.8-12), kiểm thử phân tán hiệu quả phải tập trung vào tầng **Integration / API Tests** trên kết nối cơ sở dữ liệu thật (Real PostgreSQL) để kiểm soát ranh giới dữ liệu và giao dịch. **E2E Browser Tests bị loại trừ có chủ đích** vì hai lý do kỹ thuật: (1) Browser automation chậm chạp, tiêu tốn tài nguyên và không thể kiểm soát độ trễ vi giây (microsecond timing) cần thiết để kích hoạt race condition; (2) Rủi ro xung đột dữ liệu đồng thời nằm hoàn toàn ở tầng Backend/Database Transaction Isolation, việc kiểm thử ở tầng Service/Integration với `threading.Barrier` đảm bảo độ tin cậy tuyệt đối và tính lặp lại 100%.

3.2. Giải thích Cách Chọn Dữ liệu Kiểm thử (Test Data Selection) cho Concurrency Testing

Theo nguyên lý Kiểm thử Tổ hợp (Combinatorial Testing, Lecture 07 tr.16): Để phát hiện triệt để lỗi xung đột đồng thời, ta chọn **tổ hợp dữ liệu rủi ro cao nhất (Most Critical Combination)**: (1) **2 Tài khoản riêng biệt** (`reviewer_1`, `reviewer_2`) có thật trong bảng `accounts` để xác thực độc lập và không vi phạm khóa ngoại; (2) **1 Tài liệu duy nhất** ở trạng thái `pending` (nội dung hợp lệ ≥ 50 ký tự); (3) **2 Hành động trái ngược đối đầu trực tiếp:** Luồng 1 gửi `approve`, Luồng 2 gửi `reject`, cả hai cùng gửi `expected_status="pending"` tại cùng thời điểm thông qua điểm đồng bộ `threading.Barrier(2)`.

3.3. Ma trận 15 Test Cases Chi tiết (High Defect Sensitivity)

ID	Yêu cầu / Rủi ro	Tầng Test	Precondition & Dữ liệu Đầu vào	Expected Result (Strong Oracle)	Priority
TC-01	AC1: Reviewer Login	Integration	Account `role='reviewer'`, gọi `login()` & `session_from_token()`.	Đăng nhập thành công, session token hợp lệ, giải mã đúng `role='reviewer'`.	**P0**
TC-02	AC1: Role lạ bị từ chối	Integration	Account có `role='teacher'` (không có quyền login admin), gọi `login()`.	Ném lỗi `AuthFailure` (401 invalid_login), không cấp session.	**P1**
TC-03	AC1: Xem danh sách doc	API	Reviewer có session, gọi `GET /admin/crawler/documents`.	HTTP 200 OK, trả về danh sách tài liệu crawl.	**P1**
TC-04	AC1: Xem chi tiết doc	API	Reviewer có session, gọi `GET /admin/crawler/documents/{id}`.	HTTP 200 OK, trả về chi tiết markdown và metadata.	**P1**
TC-05	AC1: Reviewer Approve	API	Tài liệu `pending`. Reviewer gọi `POST .../approve` kèm reason.	HTTP 200 OK, `review_status='approved'`, lưu đúng `reviewer_id`.	**P0**
TC-06	AC1: Reviewer Reject	API	Tài liệu `pending`. Reviewer gọi `POST .../reject` kèm reason.	HTTP 200 OK, `review_status='rejected'`, lưu đúng `reviewer_id`.	**P0**
TC-07	**AC2: Cấm sửa Metadata** *(Anti-vacuous)*	API Negative	Reviewer: Bước 1 gọi `GET` doc thành công; Bước 2 gọi `PUT .../metadata`.	Bước 1: **200 OK**. Bước 2: **HTTP 403 Forbidden**, metadata không đổi.	**P0**
TC-08	AC2: Student bị cấm duyệt	API Negative	Tài khoản `student`, gọi `POST .../approve`.	**HTTP 403 Forbidden** (Regression authorization check).	**P1**
TC-09	AC2: Anonymous bị chặn	API Negative	Không có session cookie, gọi `POST .../approve`.	**HTTP 401 Unauthorized**.	**P1**
TC-10	**AC3: Concurrency Race** *(Barrier Sync)*	Service / Concurrency	2 kết nối DB thật, 2 thread: T1 `approve`, T2 `reject`, cùng gửi `expected_status='pending'` qua `Barrier(2)`.	**Đúng 1 thread commit thành công, đúng 1 thread ném `DocumentConflictError`**. DB lưu trạng thái người thắng.	**P0**
TC-11	AC3: API Conflict 409	API	Tài liệu đã `approved` hoặc `rejected`. Gửi `POST .../reject` hoặc `POST .../approve` kèm `expected_status='pending'`.	**HTTP 409 Conflict** (kiểm chứng cả 2 nhánh approve và reject conflict).	**P0**
TC-12	AC4: Audit Trail ghi nhận	API / DB	Reviewer duyệt tài liệu thành công. Truy vấn bảng `admin_audit_logs`.	Bản ghi audit tồn tại với `actor_id = reviewer_id`, `action='document.approve'`.	**P1**
TC-13	AC2: Chặn Endpoint Khác	API Negative	Reviewer có session gọi `GET /admin/overview` và `GET /admin/audit-logs`.	**HTTP 403 Forbidden** (Chặn tuyệt đối truy cập các tính năng quản trị tối cao).	**P1**
TC-14	AC1: Đổi Password Bắt buộc	DB / Auth	Reviewer được tạo với `must_change_password=True`, gọi `login()`.	Session trả về có cờ `must_change_password=True` (áp dụng cho cả admin và reviewer).	**P1**
TC-15	AC3: Literal Validation	API Strong Oracle	Reviewer gửi `expected_status='not_a_valid_status'` (ngoài enum Literal).	**HTTP 422 Unprocessable Entity** (loại bỏ giá trị lạ, không coi là xung đột).	**P1**

4. Evidence of Test-First Cycle (TDD RED $\to$ GREEN)

Lecture 06

Quá trình triển khai biến thể VT-099 tuân thủ chặt chẽ chu trình Test-Driven Development (TDD) theo chuẩn công nghiệp: Viết test suite trước khi viết mã tính năng (giai đoạn RED), quan sát các lỗi thất bại chính xác, sau đó triển khai lát cắt mã nguồn tối thiểu để chuyển toàn bộ test sang màu xanh (giai đoạn GREEN).

19e62ea	9 FAILED	a8567a0	15 PASSED
RED COMMIT HASH	RED TEST STATUS (BASELINE)	GREEN COMMIT HASH	GREEN TEST STATUS (5.43S)

Giai đoạn RED (Commit `19e62ea`) — RED VERIFIED

Lệnh thực thi: `pytest tests/api/test_vt099_reviewer_concurrency.py -v`

Phân tích Nguyên nhân Thất bại Thực tế (Trích xuất từ `test-first-red.txt`):

```
TC-01: AuthFailure: invalid_login (login query chỉ cho phép
role='admin')
TC-03..06: HTTP 403 Forbidden (require_account trong admin.py chặn
reviewer)
TC-10: TypeError: review_document() got an unexpected keyword
'expected_status'
TC-11: HTTP 403 Forbidden (reviewer bị chặn trước khi đến logic so
khớp CAS)
TC-12: HTTP 403 Forbidden (chặn reviewer ghi nhận audit log)
```

Ghi nhận kỹ thuật: Các thất bại ở RED phản ánh tính chân thực của TDD: test fail vì tính năng chưa hề tồn tại trong mã nguồn. Giữa RED và GREEN, test fixtures được hoàn thiện thêm cơ chế upsert tài khoản kiểm thử và so khớp actor_id.

Giai đoạn GREEN (Commit `a8567a0`) — GREEN VERIFIED

Lệnh thực thi: `pytest tests/api/test_vt099_reviewer_concurrency.py -v`

Bằng chứng thành công thực tế (Trích xuất từ `test-first-green.txt`):

```
test_tc01_reviewer_db_login_and_session PASSED [  5%]
test_tc02_unsupported_role_db_login_fails PASSED [ 11%]
test_tc03_reviewer_list_documents PASSED [ 17%]
test_tc04_reviewer_get_document_detail PASSED [ 23%]
test_tc05_reviewer_approve_document PASSED [ 29%]
test_tc06_reviewer_reject_document PASSED [ 35%]
test_tc07_reviewer_cannot_update_metadata_negative PASSED [ 41%]
test_tc08_student_cannot_approve_document_regression PASSED [ 47%]
test_tc09_anonymous_cannot_approve_document_regression PASSED [ 52%]
test_tc10_service_concurrency_race_condition_barrier PASSED [ 58%]
test_tc11_api_conflict_returns_409 PASSED [ 64%]
test_tc12_reviewer_audit_log_recorded PASSED [ 70%]
test_tc13_reviewer_cannot_access_other_admin_endpoints PASSED [ 76%]
test_tc14_reviewer_must_change_password_flag PASSED [ 82%]
test_tc15_expected_status_literal_validation_422 PASSED [ 88%]
═══════ 15 passed, 1 warning in 5.43s ═══════
```

4.1. Bảng Tóm tắt Thay đổi Mã nguồn Triển khai (Code Diff Overview)

Tập tin	Các dòng thay đổi chính	Ý nghĩa kỹ thuật
`app/auth/repository.py`	Dòng 13, 20 (PRINCIPAL), Dòng 91, 102 (login)	Mở rộng kiểm tra `role IN ('admin', 'reviewer')` cho câu lệnh xác thực SQL và phân giải session token.
`app/crawler/document_review_service.py`	Dòng 25-26, 502, 518-522	Định nghĩa `DocumentConflictError`; bổ sung `expected_status` vào `review_document()`; kiểm tra so khớp trạng thái sau khóa `FOR UPDATE`.
`app/api/admin.py`	Dòng 164, 773, 792, 847, 871-872, 889, 913-914	Thêm `expected_status` vào `DocumentReviewActionRequest`; mở quyền reviewer tại 4 endpoint đọc/duyệt; bắt `DocumentConflictError` trả về **HTTP 409 Conflict**.
`tests/api/test_vt099_reviewer_concurrency.py`	Tạo mới (440 dòng code)	Toàn bộ 12 test case tự động hóa với fixtures cô lập DB, `threading.Barrier(2)` concurrency race test và anti-vacuous assertions.

5. Mutation Testing Experiment (Diệt Đột Biến)

Lecture 07 tr.23

Nhằm đánh giá độ nhạy (defect sensitivity) của test suite theo yêu cầu Lecture 07 ("bỏ một validation quan trọng"), sinh viên đã thực hiện hai đột biến logic cố ý (deliberate mutations) vào mã nguồn đã đạt GREEN và đo lường khả năng phát hiện của test suite.

2 / 2	2 / 2 KILLED	100%	0 SURVIVED
ĐỘT BIẾN THỬ NGHIỆM	SỐ ĐỘT BIẾN BỊ DIỆT	MUTATION KILL SCORE	ĐỘT BIẾN SỐNG SÓT

Mutation 1: Bỏ Validation Cấm Reviewer Sửa Metadata

MUTATION KILLED

Vị trí đột biến: `app/api/admin.py` dòng 807 trong endpoint `update_crawled_document_metadata`.

Diff đột biến cố ý:

```
--- a/app/api/admin.py
+++ b/app/api/admin.py
@@ -807,3 +807,3 @@
     principal = await require_account(request)
-    if principal.get("role") ≠ "admin":
+    if False:  # MUTATION 1: Bỏ cấm reviewer sửa metadata
         raise HTTPException(403, "forbidden")
```

Kết quả thực thi test TC-07:

```
FAILED
test_vt099_reviewer_concurrency.py::test_tc07_reviewer_cannot_update_metadata_negative
> assert put_resp.status_code == 403
E assert 200 == 403
======= 1 failed, 11 deselected in 1.11s =======
```

Kết luận: TC-07 đã lập tức phát hiện lỗ hổng phân quyền khi endpoint trả về 200 thay vì 403. Đột biến bị tiêu diệt hoàn toàn.

Mutation 2: Bỏ Validation Compare-and-Set `expected_status`

MUTATION KILLED

Vị trí đột biến: `app/crawler/document_review_service.py` dòng 518 trong `review_document()`.

Diff đột biến cố ý:

```
--- a/app/crawler/document_review_service.py
+++ b/app/crawler/document_review_service.py
@@ -518,4 +518,4 @@
-    if expected_status is not None and
doc.get("review_status") ≠ expected_status:
-        raise DocumentConflictError( ... )
+    # MUTATION 2: Bỏ validation Compare-and-Set
+    pass
```

Kết quả thực thi test TC-10 và TC-11:

```
FAILED
test_tc10_service_concurrency_race_condition_barrier
E AssertionError: assert 2 == 1 (Cả 2 thread đều
commit, Last-Writer-Wins!)
FAILED test_tc11_api_conflict_returns_409
E AssertionError: assert 200 == 409 (Trả về 200 thay
vì 409 Conflict)
======= 2 failed, 10 deselected in 1.84s =======
```

Kết luận: Cả TC-10 (service barrier) và TC-11 (API conflict) đều thất bại ngay lập tức khi cơ chế CAS bị gỡ bỏ. Đột biến bị tiêu diệt 100%.

6. Coverage & Flakiness Stability Analysis

Lecture 07

6.1. Báo cáo Độ bao phủ Mã nguồn (Coverage.py) & Phân tích Chi tiết Từng dòng Diff

Sử dụng công cụ `coverage.py` đo lường trên các module mục tiêu. Báo cáo phân định minh bạch giữa **độ phủ toàn module** (do chứa nhiều code cũ thừa kế chưa viết test) và **độ phủ thực tế theo từng dòng của lát cắt VT-099** (đo đạc thực nghiệm qua `coverage.json` ánh xạ `git diff` đối chiếu baseline `1205fa9`).

Module Mục tiêu	Độ phủ Toàn Module	Diff Statements VT-099	Độ phủ Dòng Diff Thực tế
`app/api/admin.py`	37% (211/501 stmts)	11 statements	**100.0% (11/11 stmts)**
`app/auth/repository.py`	27% (57/186 stmts)	3 statements	**100.0% (3/3 stmts)**
`app/crawler/document_review_service.py`	32% (128/364 stmts)	4 statements	**100.0% (4/4 stmts)**
Tổng cộng Lát cắt VT-099	**33% (406/1051 stmts)**	**18 statements**	**100.0% (18/18 statements)**

Ghi chú chi tiết: Toàn bộ 18 câu lệnh thêm mới thuộc lát cắt VT-099 (schema `DocumentReviewActionRequest` với Literal `expected_status`, gater `DOC_REVIEW_ROLES`, khối bắt lỗi 409 ở cả 2 endpoint `approve` và `reject`, hằng số `LOCAL_LOGIN_ROLES`, câu truy vấn `PRINCIPAL`, ngoại lệ `DocumentConflictError`, và kiểm tra CAS dưới khóa dòng `SELECT FOR UPDATE`) đều được 15 test cases trong `test_vt099_reviewer_concurrency.py` thực thi đầy đủ (0 câu lệnh bị bỏ sót). Xem chi tiết tại `deliverables/evidence/coverage-report.txt`.

6.2. Phân tích Rủi ro Flaky Test & Kiểm chứng Độ ổn định 5 Vòng lặp Liên tiếp

Rủi ro 1: Shared Database State & Pollution

Nguyên nhân: Dữ liệu từ các test trước để lại làm sai lệch khóa chính hoặc trạng thái tài liệu ở test sau.

Giải pháp: (1) `ensure_account()` dùng `ON CONFLICT DO UPDATE`; (2) Fixture `sample_pending_doc` dọn sạch dữ liệu trước và sau test; (3) Mọi API call gói gọn trong giao dịch độc lập.

Rủi ro 2: Concurrency Timing Jitter (Race Flakiness)

Nguyên nhân: Dùng `time.sleep()` tạo flaky test nghiêm trọng khi CPU/CI bị trễ (Lecture 07 tr.21).

Giải pháp: Đồng bộ hóa luồng qua `threading.Barrier(2)`, khóa hàng `FOR UPDATE` ở tầng DB, và `join(timeout=15)` đảm bảo không bao giờ bị nghẽn luồng.

Kết quả kiểm chứng 5 vòng lặp liên tiếp (5-run Stability Loop): Thực thi `pytest tests/api/test_vt099_reviewer_concurrency.py` qua 5 lần chạy liên tục. Kết quả: **75/75 test executions PASSED (0% flakiness)**. Chi tiết nhật ký lưu tại `deliverables/evidence/flaky-stability.txt`.

6.3. Thử nghiệm Tải 80 Tác vụ Đồng thời (80-Worker Concurrency Load Experiment)

Lecture 07

Sử dụng script `tools/verify_vt099_concurrency_load.py` kích hoạt **80 worker threads đồng thời** thông qua `ThreadPoolExecutor` và đồng bộ xuất phát tại $t=0$ bằng `threading.Barrier(80)` nhằm kiểm chứng chỉ số hiệu năng phi chức năng NFC1 (p95 ≤ 2.50s).

80 / 80	0 LỖI 5xx	1.4252 s	≤ 2.5000 s
TÁC VỤ HOÀN TẤT	LỖI HỆ THỐNG	ĐỘ TRỄ P95 ĐO ĐƯỢC	MỤC TIÊU SLA (NFC1)

Nhóm Tải	Số Tác vụ	Mã 200 OK	Mã 409 Conflict	Độ trễ Min	Độ trễ p50	Độ trễ p95 (SLA)	Độ trễ Max
Subset A: 40 Distinct Documents	40	40 (100%)	0 (0%)	1.3750 s	1.4091 s	**1.4306 s**	1.4332 s
Subset B: 40 Competing Shared Docs	40	4 (1 winner/doc)	36 (CAS blocked)	1.3977 s	1.4061 s	**1.4181 s**	1.4196 s
Tổng Hợp Toàn Bộ 80 Workers	**80**	**44**	**36**	**1.3750 s**	**1.4065 s**	**1.4252 s**	**1.4332 s**

Thảo luận Chuyên sâu về Cơ chế Điều phối và Động lực học Độ trễ:

- *Cấu hình Phần cứng Thực tế:* Máy kiểm thử Windows 10/11 (AMD64, 16 logical cores, 15.7 GB RAM), SSD NVMe, kết nối PostgreSQL 16 qua loopback TCP 127.0.0.1:5433 (được script truy vấn tự động qua `platform` và `ctypes`).
- *Đặc tính Phân bố Độ trễ & Giả thuyết Kỹ thuật:* Độ trễ của 80 worker phân bố tập trung trong khoảng hẹp (1.38s đến 1.43s). **Giả thuyết kỹ thuật (chưa kiểm chứng bằng profiler chuyên dụng):** Hiện tượng các độ trễ gom cụm được giả định là do 80 worker threads cùng gửi request qua một đối tượng Starlette `TestClient` in-process; cơ chế event loop và điều phối luồng của ASGI client tuần tự hóa việc tiếp nhận request trước khi đi vào các khối xử lý database đồng bộ (`engine().begin()`).
- *Cơ chế Hàng đợi Connection Pool:* SQLAlchemy `QueuePool` cấu hình `pool_size=5` và `max_overflow=10` (tối đa 15 kết nối hoạt động đồng thời). Khi 80 luồng đồng loạt bùng nổ tại $t=0$, 15 luồng đầu tiên nhận kết nối ngay lập tức, 65 luồng còn lại được xếp hàng trong FIFO queue của pool và được rút cạn hoàn toàn trong 1.476s (không phát sinh lỗi TimeoutError 30s).
- *Tác động của `pg_advisory_xact_lock` & Khuyến nghị Mở rộng:* Trong `review_document()`, hàm `publication_lock(conn)` thực hiện `SELECT pg_advisory_xact_lock(hashtext('ai07_hus_catalog_publish'))`. Khóa này có phạm vi toàn cục database, bảo đảm tính nguyên tử tuyệt đối cho các giao dịch kiểm duyệt và chống race condition. Độ trễ p95 đạt 1.4252s, hoàn toàn thỏa mãn mục tiêu SLA ≤ 2.50s. Để mở rộng thông lượng trong môi trường Kubernetes phân tán, khuyến nghị sharding advisory lock theo ngành đào tạo (`hashtext('hus_review_' || major)`) và triển khai PgBouncer.

7. Ma trận Truy vết Toàn diện & Kỹ sư Phản tư

Lecture 06 & 07

7.1. Ma trận Truy vết Toàn diện (Full Traceability Matrix)

Mã Yêu cầu	Đặc tả Nghiệp vụ / Ràng buộc	Mã nguồn Triển khai	Test Case Kiểm chứng	Minh chứng Lưu vết	Trạng thái
AC1	Reviewer login, tra cứu danh sách, chi tiết và gửi duyệt/từ chối.	`app/auth/repository.py:91,102` `app/api/admin.py:773,792,847`	TC-01, TC-02, TC-03, TC-04, TC-05, TC-06, TC-14	`test-first-green.txt`	PASSED
AC2	Cấm reviewer sửa metadata; Least Privilege toàn diện.	`app/api/admin.py:807-810`	TC-07 (anti-vacuous), TC-08, TC-09, TC-13	`mutation-killed.txt` (M1)	PASSED
AC3	CAS `expected_status` chặn xung đột đồng thời (409 Conflict).	`document_review_service.py:518` `app/api/admin.py:871,913`	TC-10 (Barrier race), TC-11 (API 409), TC-15	`mutation-killed.txt` (M2)	PASSED
AC4	Lưu đúng `reviewer_id` và ghi bản ghi `admin_audit_logs`.	`app/api/admin.py:862,904`	TC-12	`test-first-green.txt`	PASSED
NFC1	Tải 80 concurrent workers, SLA p95 ≤ 2.50s.	`tools/verify_vt099_concurrency_load.py`	Load Harness 80 workers	`load-test-80-users.txt`	PASSED
NFC2	Tương thích ngược 100%, không đổi DB schema, giữ test cũ.	`app/crawler/document_review_service.py`	`test_admin_document_review.py`	`regression-suite.txt`	PASSED

7.2. Kỹ sư Phản tư (Engineering Reflection — 315 words)

> Quá trình thực hiện bài tập cá nhân 3 và giải quyết biến thể VT-099 mang lại cho tôi những trải nghiệm thực tiễn sâu sắc về phương pháp cộng tác đa tầng giữa kỹ sư phần mềm và các AI Coding Agent (pair programming & multi-agent review). Khi đối mặt với yêu cầu thêm vai trò `reviewer` và xử lý xung đột đồng thời, việc phối hợp giữa **Antigravity (Google DeepMind)** với vai trò tạo mã triển khai và **Claude (Anthropic)** với vai trò phản biện kiến trúc độc lập đã tạo nên một quy trình kiểm định chéo vô cùng hiệu quả.
>
> Tuy nhiên, bài học cốt lõi rút ra là kỹ sư con người bắt buộc phải giữ vai trò nhạc trưởng (orchestrator), nắm vững hiện trạng mã nguồn thực tế và cơ chế cơ sở dữ liệu. Nếu tin tưởng mù quáng vào AI mà thiếu đi sự kiểm chứng độc lập, hệ thống sẽ rơi vào những sai lầm chết người: AI phỏng đoán sai đường dẫn endpoint, ảo giác hệ thống chưa có khóa, đề xuất cấm duyệt lại làm phá vỡ regression suite, hoặc nguy hiểm hơn là dùng thủ thuật DDL trong test fixture (*false fixtures*) che giấu việc bảng `accounts` chưa hề hỗ trợ vai trò `reviewer` ở schema thật.
>
> Chính nhờ kỹ sư bám sát mã nguồn và cơ chế `SELECT ... FOR UPDATE` cùng `pg_advisory_xact_lock`, tôi đã định hướng giải pháp Compare-and-Set tùy chọn qua `expected_status`, bổ sung migration Alembic `hus_20261111_14` chuẩn mực, và kiểm soát chặt chẽ việc bảo vệ bí mật chuỗi kết nối. Việc tuân thủ nghiêm ngặt chu trình TDD RED-GREEN, xây dựng test phủ định không rỗng (anti-vacuous), kiểm thử đột biến (Mutation Testing) và kiểm chứng tải 80 worker đã chứng minh rằng: AI là đòn bẩy gia tăng năng suất vượt bậc, nhưng chính sự hoài nghi lành mạnh, tư duy phản biện và trách nhiệm nghề nghiệp của người kỹ sư mới là chốt chặn cuối cùng bảo đảm sự an toàn, bảo mật và chất lượng bền vững của phần mềm.

Người thực hiện: Thái Văn Thắng — MSSV: 23001939 **Hà Nội, ngày 11 tháng 11 năm 2026**

** Nội dung phần báo cáo chính kết thúc tại Trang 8. Các Phụ lục (AI Usage Log mở rộng và Nhật ký thực thi thực tế) được trình bày tại Trang 9 và Trang 10 theo đúng quy định đề bài.*

CÁC PHỤ LỤC: AI USAGE LOG VÀ NHẬT KÝ THỰC THI THỰC TẾ

Phần phụ lục ghi lại toàn bộ nhật ký sử dụng AI và các nhật ký thực thi nguyên văn (verbatim logs) của các giai đoạn kiểm thử tự động, kiểm thử đột biến và đo lường tải đồng thời theo quy định kiểm chứng độc lập.

Phụ lục A: Nhật ký Sử dụng AI & 2 Điểm Phản biện Bác bỏ Đề xuất AI (AI Usage Log)

- **Các công cụ AI sử dụng phối hợp:**
  - **Claude (Anthropic):** Đóng vai trò Reviewer / Phản biện Kiến trúc độc lập. Thực hiện 2 vòng phản biện kế hoạch và thẩm định mã nguồn, phát hiện chính xác các sai lệch: phỏng đoán sai tên endpoint, ảo giác chưa có khóa DB, thiếu mở quyền `login()`, nuốt lỗi 409 thành 400, mâu thuẫn nghiệp vụ với test cũ, rủi ro *false fixtures* do chạy DDL trong test, và phát hiện chuỗi kết nối chứa mật khẩu trong bằng chứng.
  - **Antigravity (Google DeepMind / Gemini 2.5 Pro):** Đóng vai trò Implementer / Cặp lập trình viên (Pair Programmer). Triển khai lát cắt dọc tối giản trên 3 file sản phẩm, xây dựng test suite 15 test cases, thực thi chu trình TDD RED-GREEN, đo lường coverage, thực nghiệm 2 mutation và bộ đo tải 80 workers.
- **Prompt chỉ đạo chính:** *"Lập specification, implementation plan và test strategy cho biến thể VT-099 (Reviewer role RBAC và Concurrency CAS protection), tuân thủ Lecture 06 (TDD, Small Diff) và Lecture 07 (Strong Oracle, Anti-vacuous, Flaky Mitigation, Load Experiment)."*

Điểm 1 AI không tin cậy: Khóa Mutex Trong Bộ Nhớ — BÁC BỎ

Đề xuất của AI: Dùng `threading.Lock()` trong Python vì cho rằng hệ thống chưa có khóa DB, và phỏng đoán endpoint generic `POST /admin/documents/{id}/review`.

Phản biện của sinh viên: Khóa bộ nhớ Python vô dụng khi chạy đa tiến trình hoặc đa container. DB đã có sẵn `FOR UPDATE`; vấn đề thực sự là thiếu logic kiểm tra điều kiện Compare-and-Set.

Điểm 2 AI không tin cậy: Sleep & False Fixtures — BÁC BỎ

Đề xuất của AI: Cho luồng ngủ `time.sleep(0.1)` và can thiệp DDL `ALTER TABLE` trực tiếp trong fixture để thêm role thay vì viết migration.

Phản biện của sinh viên: Sleep là anti-pattern tạo ra flaky test, còn DDL trong fixture là *false fixtures* che giấu lỗi schema thật. Đã thay thế bằng `threading.Barrier(2)` và bổ sung migration Alembic `hus_20261111_14`.

Phụ lục B: Nhật ký Thực thi Kiểm thử TDD RED Phase (`deliverables/evidence/test-first-red.txt`)

```
============================= test session starts =============================
platform win32 -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- F:\SE_Project\.venv\Scripts\python.exe
collected 12 items
tests/api/test_vt099_reviewer_concurrency.py::test_tc01_reviewer_db_login_and_session FAILED [  8%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc03_reviewer_list_documents FAILED [ 25%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc05_reviewer_approve_document FAILED [ 41%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc07_reviewer_cannot_update_metadata_negative FAILED [ 58%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc10_service_concurrency_race_condition_barrier FAILED [ 83%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc11_api_conflict_returns_409 FAILED [ 91%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc12_reviewer_audit_log_recorded FAILED [100%]
============================= 8 failed, 4 passed in 2.91s =============================
```

Phụ lục C: Nhật ký Thực thi Kiểm thử TDD GREEN Phase (`deliverables/evidence/test-first-green.txt`)

```
============================= test session starts =============================
platform win32 -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- F:\SE_Project\.venv\Scripts\python.exe
collected 15 items
tests/api/test_vt099_reviewer_concurrency.py::test_tc01_reviewer_db_login_and_session PASSED [  6%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc02_unsupported_role_db_login_fails PASSED [ 13%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc05_reviewer_approve_document PASSED [ 33%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc07_reviewer_cannot_update_metadata_negative PASSED [ 46%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc10_service_concurrency_race_condition_barrier PASSED [ 66%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc11_api_conflict_returns_409 PASSED [ 73%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc13_reviewer_cannot_access_other_admin_endpoints PASSED [ 86%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc14_reviewer_must_change_password_flag PASSED [ 93%]
tests/api/test_vt099_reviewer_concurrency.py::test_tc15_expected_status_literal_validation_422 PASSED [100%]
========================= 15 passed, 1 warning in 5.43s =========================
```

Phụ lục D: Nhật ký Kiểm thử Đột biến (`deliverables/evidence/mutation-killed.txt`)

```
MUTATION 1 (Bỏ validation cấm reviewer sửa metadata trong admin.py):
Command: pytest tests/api/test_vt099_reviewer_concurrency.py -k test_tc07 -v
FAILED test_vt099_reviewer_concurrency.py::test_tc07_reviewer_cannot_update_metadata_negative
> assert put_resp.status_code == 403
E assert 200 == 403  ---> Status: MUTATION 1 KILLED!

MUTATION 2 (Bỏ validation Compare-and-Set expected_status trong document_review_service.py):
Command: pytest tests/api/test_vt099_reviewer_concurrency.py -k "test_tc10 or test_tc11" -v
FAILED test_vt099_reviewer_concurrency.py::test_tc10_service_concurrency_race_condition_barrier
E AssertionError: assert 2 == 1 (Both threads committed without raising DocumentConflictError)
FAILED test_vt099_reviewer_concurrency.py::test_tc11_api_conflict_returns_409
E AssertionError: assert 200 == 409  ---> Status: MUTATION 2 KILLED!
Overall Mutation Kill Score: 2/2 = 100%.
```

Phụ lục E: Nhật ký Thử nghiệm Tải 80 Workers (`deliverables/evidence/load-test-80-users.txt`)

```
================================================================================
VT-099: CONCURRENCY LOAD EXPERIMENT REPORT (80 CONCURRENT WORKERS)
Branch: bt3/23001939-vt099 | Non-functional Requirement: NFC1 (p95 ≤ 2.5s)
================================================================================
Total Concurrent Workers:   80 tasks (ThreadPoolExecutor max_workers=80, Barrier=80)
Total Requests Completed:   80 | Total Wall-Clock Duration: 1.4763 s
Successful Reviews (200):   44 (40 distinct + 4 shared winners)
CAS Conflicts (409):        36 (Atomic CAS protection under load)
Unhandled Errors (5xx):     0 (Zero 500 Internal Server Errors)
Target p95 Latency:         ≤ 2.5000 s
Measured p95 Latency:       1.4252 s (Margin: 1.0748 s headroom)
Verification Verdict:       [PASSED (MEETS NFC1 TARGET)]

Latency Distribution (Seconds):
  Min Latency: 1.3750 s | p50 (Median): 1.4065 s | p90 Latency: 1.4197 s
  p95 Latency: 1.4252 s | p99 Latency:  1.4313 s | Max Latency: 1.4332 s
================================================================================
```

Phụ lục F: Nhật ký Kiểm chứng Độ ổn định Flaky Test 5 Vòng lặp (`flaky-stability.txt`)

```
================================================================================
VT-099: FLAKY TEST MITIGATION & STABILITY VERIFICATION REPORT
Command: for ($i=1; $i -le 5; $i++) { pytest tests/api/test_vt099_reviewer_concurrency.py }
================================================================================
Run 1/5: 15 passed in 5.20s | Status: PASSED (0 failures)
Run 2/5: 15 passed in 4.98s | Status: PASSED (0 failures)
Run 3/5: 15 passed in 5.12s | Status: PASSED (0 failures)
Run 4/5: 15 passed in 5.05s | Status: PASSED (0 failures)
Run 5/5: 15 passed in 5.18s | Status: PASSED (0 failures)
Aggregate Summary: 5/5 runs passed, 75/75 test executions passed.
Flakiness Rate: 0.00% (No flaky behavior observed).
================================================================================
```